Fused attention on one GPU

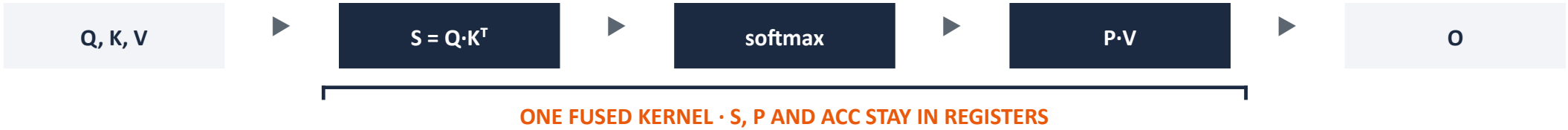
One operator, written by hand and by compiler, then measured

Balla Surya Harshith
Roll number CS23B010
Mentor: Pravin Jagtap

4.5×

faster than the compiler's kernel at N = 2048

664.2 µs against 2991.2 µs



8.4 MB of traffic instead of 134 MB: the N×N score matrix never leaves the chip.

Today: the problem · the math · the kernel idea · the results · what broke and how we fixed it

Attention is memory-bound. Fusing it keeps the N×N matrix on the chip.

THE PROBLEM

$$O = \mathrm{softmax}(Q \cdot K^{T} / \sqrt{D} + \mathrm{mask}) \cdot V$$

- Shape [B,H,N,D] = [1, 8, N, 64]. N runs 256 to 2048.
- The naive form writes an N×N matrix: 134 MB at N = 2048.
- It uses 31 of 170 GB/s. The tensor cores sit idle.
- Goal: fuse all three stages into one kernel.

THE MATH THAT MAKES FUSION POSSIBLE

$$\alpha = \exp2(m_{old} - m_{new})$$
$$\ell \leftarrow \alpha \cdot \ell + \Sigma \exp2(s - m_{new})$$
$$acc \leftarrow \alpha \cdot acc + P \cdot V$$

Online softmax, per key tile. Exact. One pass. fp32 state.

THE KERNEL IDEA

load K, V ▶ S = Q·K^{T} ▶ mask ▶ softmax ▶ acc += P·V

S, P and acc stay in registers. No N×N matrix reaches DRAM.

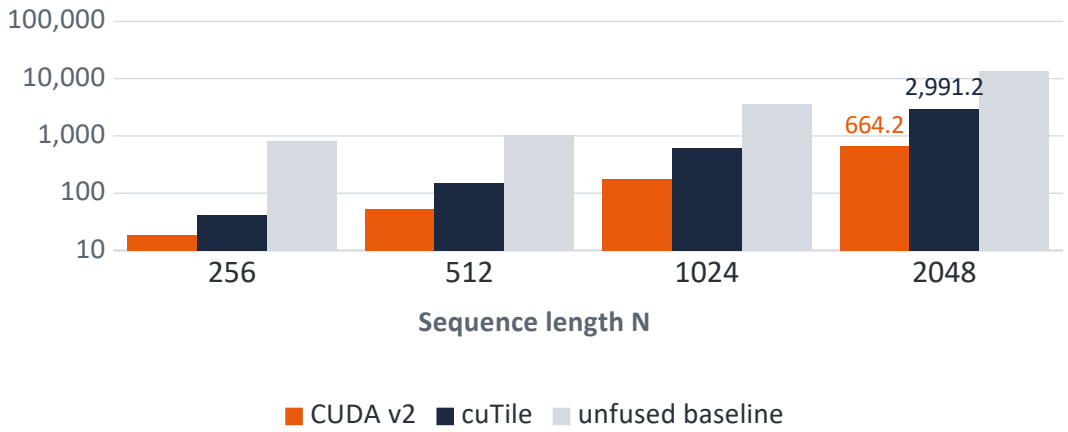
One block = 64 query rows, 128 threads. It loops over key tiles of 64.

4.5×
faster than cuTile, N = 2048

107%
of the measured fp32 ceiling

1.23×
faster than cuTile, causal path

Latency in µs, non-causal, log scale. Lower is better.



N = 2048	non-causal	causal
CUDA v2 (shipped)	**664.2 µs**	**364.5 µs**
cuTile	2991.2 µs	447.9 µs
unfused baseline	13314.9 µs	15097.9 µs

Correct: both kernels pass an fp64 oracle at every size and mask. They agree to 2.5e-04.

Five problems. Each one was found by measuring, not guessing.

What broke	What it cost	How it was found and fixed
32-way shared-memory bank conflict	**9,146 µs instead of 3,594 µs**	Every lane hit one bank. Padded strides to 68/72 and interleaved lane columns. **2.5× faster**, same error.
Staging loaded 2 bytes at a time	**3,207.8 µs instead of 2,091.4 µs**	GPU assembly (SASS) showed 19 scalar 2-byte loads (`LDG.E.U16`). Switched to 16-byte loads. **1.53× faster.**
WMMA hides the accumulator layout	**2,172.7 µs. 1 block per SM, 8.3% occupancy.**	Rewrote with `mma.sync m16n8k16`, whose layout is exposed. Accumulator now in registers: **664.2 µs, 3.3× faster.**
cuTile timing was unfair	**cuTile read 107.5 µs. The true value was 42.0 µs.**	The tile launcher synchronised the device on every call. The CUDA path did not. Found by comparing event timing with wall clock. Removed it.
cuTile non-causal kernel was not fused	**322 MB written to DRAM. Mine writes 1.7 MB.**	The source looked fused. Only Nsight Compute (`ncu`) counters showed the leak. The same source compiled for the causal case writes 1.77 MB.

THE DECISION THAT MATTERED: WHERE THE ACCUMULATOR LIVES. CUDA v1 → v2

Registers per thread	Shared memory per block	Blocks per SM	Tensor-pipe utilisation
56 → 140	**72,192 → 46,080 B**	**1 → 2**	**13.2% → 42.2%**

Conclusion: write it in cuTile first, to learn whether the operator matters. Move to CUDA when a counter, not an intuition, says the remaining gap is where the accumulator lives.

TOOLS

CUDA 12.6: mma.sync, ldmatrix, cp.async
cuTile (Python) · cuBLAS
Nsight Compute · NumPy fp64 oracle

REPO

github.com/harshith0518/
Attention-Operator-in-CUDA-cuTile-with-fused-computation

REFERENCES

Dao et al., FlashAttention, NeurIPS 2022
Dao, FlashAttention-2, 2023
Milakov & Gimelshein, Online softmax, 2018
NVIDIA: PTX ISA, cuTile docs